

## TECHNICAL TRAINING & ASSESSMENT PROJECT

# Reservio

## Multi-Business Booking Platform

Backend Engineering Project for Ahmed

### Primary Goal

Design and build a reusable booking engine that can serve different business types - such as clinics, salons, restaurants, sports centers, trainers, or meeting-room providers - without hard-coding the booking logic to one specific industry.

|                   |                                                     |
|-------------------|-----------------------------------------------------|
| Project Type      | Training + Final Technical Assessment               |
| Primary Stack     | Node.js / TypeScript / Fastify / PostgreSQL / Redis |
| Execution         | Fully local with Docker Compose                     |
| First Deliverable | Development Plan + Technical Design (before coding) |

**Important: Do not begin implementation until the Development Plan and Technical Design have been reviewed.**

# 1. Project Overview

You will build a backend platform for businesses that offer time-based reservations. The system must be generic enough to support multiple business models while keeping one consistent booking engine.

- Clinic -> a customer books a doctor for a consultation.
- Salon -> a customer books a barber or stylist for a service.
- Sports center -> a customer books a court, field, or room.
- Restaurant -> a customer reserves a table or reservable area.
- Trainer / coach -> a customer books a session with a person.
- Coworking space -> a customer books a meeting room.

## Core Domain Principle

The booking engine must not depend on a specific business type. Avoid spreading logic such as `if business.type === "clinic"` or `if business.type === "salon"` throughout the booking service. Business types may be metadata or configuration, but the core scheduling and reservation logic must remain generic.

# 2. Learning Objectives

This project is designed to test whether you can move beyond simple CRUD APIs and design a backend system that handles real business rules, data integrity, concurrency, caching, asynchronous work, testing, and documentation.

- Translate business requirements into a clean relational domain model.
- Design a maintainable Node.js / TypeScript backend structure.
- Use PostgreSQL correctly for transactional and concurrency-sensitive operations.
- Use Redis where it provides real value, not as a decorative dependency.
- Implement authentication, authorization, validation, and safe error handling.
- Generate and validate bookable time slots.
- Prevent overlapping or duplicate reservations under concurrent load.
- Use queues/background workers for delayed and asynchronous actions.
- Write unit, integration, API, and concurrency tests.
- Deliver a system that another developer can clone and run locally.

# 3. Required Technology Baseline

| Area    | Required / Recommended | Purpose                              |
|---------|------------------------|--------------------------------------|
| Runtime | Node.js + TypeScript   | Primary backend runtime and language |

| Area           | Required / Recommended            | Purpose                                                     |
|----------------|-----------------------------------|-------------------------------------------------------------|
| HTTP framework | Fastify                           | API routing, plugins, validation, performance               |
| Database       | PostgreSQL                        | Authoritative relational data store                         |
| ORM            | Prisma                            | Schema, migrations, query layer                             |
| Cache          | Redis                             | Short-lived availability/slot cache and supporting concerns |
| Queue          | BullMQ                            | Background jobs and delayed work                            |
| Auth           | JWT                               | Access control and authenticated API usage                  |
| Validation     | Zod or Fastify JSON Schema        | Request and domain validation                               |
| Documentation  | Swagger / OpenAPI                 | Interactive API documentation                               |
| Testing        | Vitest or Jest + API test tooling | Automated validation of behavior                            |
| Environment    | Docker + Docker Compose           | Reproducible local execution                                |

## 4. Core Domain Model

You are responsible for proposing the final schema. The following concepts define the minimum domain that your design must support.

**Business:** The organization offering reservations. Example: clinic, salon, sports center, restaurant.

**Location:** A physical branch or place where services/resources are available.

**Service:** Something a customer can book. It includes at least duration, price/currency, and status.

**Resource:** The person or asset being reserved: doctor, barber, court, table, room, trainer, etc.

**Service-Resource Assignment:** Defines which resources can perform or provide which services.

**Availability Rule:** Recurring weekly working schedule for a resource or reservable scope.

**Availability Exception:** A date-specific override such as leave, closure, or restricted hours.

**Booking:** The customer reservation and the authoritative record of the selected time range.

**User / Customer:** Identity and ownership of bookings.

**Audit Event:** Trace of important administrative or operational changes.

#### Business

- > Location(s)
- > Service(s)
- > Resource(s)
  - > Availability
  - > Exceptions
  - > Booking(s)

## 5. Functional Requirements

### 5.1 Authentication and Users

- Registration and login.
- Secure password hashing.
- JWT access token and refresh-token strategy.
- Logout / token invalidation strategy.
- At minimum, roles: `admin` and `customer`. `staff` is optional.
- Role-based authorization for protected operations.

### 5.2 Business and Location Management

- Admin can create, read, update, activate/deactivate businesses.
- A business can have one or more locations.
- Business type can exist as metadata, but core booking logic must remain generic.
- Soft delete or an equivalent safe deactivation strategy is preferred for operational records.

### 5.3 Service Management

- Admin can create and manage bookable services.
- A service contains at least name, description, duration in minutes, price, currency, and status.
- Duration and price used for a booking must be authoritative on the server.
- Consider what historical data must be snapshotted when a service later changes.

### 5.4 Resource Management

- Admin can create resources such as people, rooms, tables, courts, or equipment.
- Resources can be assigned to one or more services.
- A resource can be disabled without deleting historical bookings.
- The model should not require a different table for every resource type.

### 5.5 Recurring Availability

- Define weekly working hours for resources or the booking scope you choose.
- Support unavailable days.
- Support more than one working period in a day if your design allows it (for example 09:00-12:00 and 14:00-18:00).
- Explain timezone handling in your design.

## 5.6 Availability Exceptions

- Support date-specific exceptions such as a day off or custom hours.
- Exceptions must override the normal recurring schedule.
- The design must clearly explain conflict/precedence rules.

## 5.7 Slot Generation

- The API must be able to return available slots for a service/resource/date.
- Slots must consider service duration, working hours, exceptions, existing bookings, and time boundaries.
- The algorithm must not return a slot that cannot fit the complete service duration.
- Document your slot granularity strategy (for example 15-minute increments) and why you chose it.

## 5.8 Booking Creation

- Customer selects a service, resource, date/start time, and any required booking data.
- Server calculates end time from the authoritative service duration.
- Server validates that the requested slot is currently bookable.
- Server must reject overlaps and out-of-hours requests.
- The booking must belong to the authenticated customer unless an explicit admin workflow is implemented.

## 5.9 Booking Lifecycle

- At minimum support: pending, confirmed, cancelled, completed, no\_show (you may add expired if your workflow requires it).
- State transitions must be explicit and validated.
- Invalid backwards transitions must be rejected.
- Cancellation must release availability for future searches.

## 5.10 Cancellation Policy

- Implement at least one configurable or clearly defined cancellation rule, such as cancellation allowed until two hours before start time.
- The rule must be enforced on the server.
- Document how admin overrides, if any, would work.

# 6. Critical Requirement: Prevent Double Booking

This is a mandatory assessment area. A correct solution must remain safe when two or more booking requests arrive at nearly the same time.

Resource: Court 1

Requested slot: 18:00 - 19:00

Request A -> arrives at the same time

Request B -> arrives at the same time

Expected result:

Exactly one booking succeeds.

The competing request receives a conflict response.

### **Do not rely on a simple read-then-write check**

A sequence such as `check availability -> if free -> insert booking` is not sufficient by itself under concurrency. Your Development Plan must explain the database-level protection strategy you will use (for example transaction/locking/constraint-based protection) and how you will test it.

## **7. Redis, Background Jobs, and Idempotency**

### **7.1 Redis**

- Use Redis for a meaningful purpose such as short-lived caching of calculated availability/slots.
- Define TTL behavior.
- Invalidate or refresh affected cache entries after booking creation, cancellation, schedule change, resource change, or relevant service-duration change.
- PostgreSQL remains the source of truth for booking integrity. Redis must not be the only mechanism preventing double booking.

### **7.2 Background Worker**

- Use BullMQ with a separate worker process/container.
- Implement at least one useful background workflow: booking reminder, pending-booking expiration, mock notification, or another justified task.
- External email/SMS services are not required. Notifications can be logged or stored in a local table for verification.
- Jobs should be retry-safe and should not create duplicated side effects.

### **7.3 Idempotency**

- `POST /bookings` must support an `Idempotency-Key` or an equivalent documented mechanism.
- Retrying the exact same booking request must not create duplicate bookings.
- Define the scope, retention, and response behavior for repeated keys.

## **8. API, Validation, Security, and Error Contract**

- Use a versioned REST API structure.

- Validate path parameters, query parameters, and request bodies.
- Do not trust client-supplied duration, final price, end time, role, or ownership fields when the server can derive them.
- Use parameterized ORM/database access and avoid raw unsafe SQL.
- Do not expose stack traces, SQL, secrets, internal file paths, or raw exceptions in API responses.
- Use environment variables for secrets and provide an `.env.example` without real secrets.
- Apply reasonable rate limiting to sensitive endpoints such as login and booking creation.
- Document authorization rules for every write endpoint.

```
{
  "success": false,
  "error": {
    "code": "BOOKING_SLOT_CONFLICT",
    "message": "The selected time slot is no longer available."
  }
}
```

## 9. Listing, Search, Filtering, and Pagination

Provide practical list endpoints rather than returning unbounded collections. At minimum demonstrate pagination, filtering, and sorting on appropriate resources.

- Businesses: name/type/status/location-oriented filters as appropriate.
- Services: business/location/status/price or other justified filters.
- Bookings: customer/status/date/resource filters with authorization-aware visibility.
- Choose and document database indexes for the most important query patterns.

## 10. Testing Requirements

Automated tests are part of the project, not an optional cleanup step. Your test suite should prove business behavior, not only that individual functions execute.

| Test Level  | Minimum Focus               | Examples                                             |
|-------------|-----------------------------|------------------------------------------------------|
| Unit        | Pure business logic         | slot calculation, state transitions, date/time rules |
| Integration | Database + Redis behavior   | repositories, transactions, cache invalidation       |
| API         | HTTP contracts              | auth, validation, permissions, booking flows         |
| Concurrency | Race-condition protection   | multiple simultaneous requests for one slot          |
| Regression  | Previously fixed edge cases | overlap boundaries, exceptions, cancellation release |

| Test Level                                                                                                                                                                                                                                                                        | Minimum Focus | Examples |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|----------|
| <b>Mandatory concurrency test</b><br>Create a test that sends multiple concurrent requests for the same resource and slot. The expected outcome must demonstrate that the persistence layer never creates overlapping confirmed/pending reservations beyond the rule you defined. |               |          |

## 11. Local Environment and Docker Requirements

The completed system must be runnable without cloud services or paid external integrations. A reviewer should be able to clone the repository, configure environment values, start the stack, run migrations/seeds, and test the API locally.

```
docker compose up --build
```

Expected local services:

- API container
- PostgreSQL
- Redis
- Background worker

Optional:

- database admin UI or test tooling

- Provide health checks where practical.
- Provide repeatable database migration and seed commands.
- Seed enough example data to demonstrate at least three different business domains.
- Swagger/OpenAPI must be available locally.
- README must contain exact setup and test commands.

## 12. Minimum Demonstration Scenarios

Your seed data and final demonstration should prove that one generic engine works across different domains. At minimum include three of the following scenarios.

| Business      | Service              | Resource    | Duration |
|---------------|----------------------|-------------|----------|
| Clinic        | General Consultation | Dr. Sara    | 30 min   |
| Salon         | Haircut              | Barber John | 45 min   |
| Sports Center | Tennis Court Booking | Court 2     | 60 min   |
| Restaurant    | Table Reservation    | Table 12    | 90 min   |

## 13. Required Deliverables

| #  | Deliverable                                | Expectation                                                              |
|----|--------------------------------------------|--------------------------------------------------------------------------|
| 01 | <b>Development Plan + Technical Design</b> | Must be submitted before implementation.                                 |
| 02 | <b>Source Code Repository</b>              | Clean commit history and meaningful structure.                           |
| 03 | <b>Database Schema + Migrations</b>        | Prisma schema and reproducible migrations.                               |
| 04 | <b>Docker Compose Environment</b>          | API, PostgreSQL, Redis, worker.                                          |
| 05 | <b>Seed Data</b>                           | Multiple business domains for local testing.                             |
| 06 | <b>OpenAPI / Swagger</b>                   | Documented endpoints and schemas.                                        |
| 07 | <b>Automated Test Suite</b>                | Unit, integration, API, concurrency.                                     |
| 08 | <b>README</b>                              | Setup, architecture summary, commands, assumptions.                      |
| 09 | <b>Final Technical Report</b>              | What was built, major decisions, known limitations, future improvements. |

## 14. First Assignment: Submit Your Development Plan

### STOP BEFORE CODING

Your first task is not implementation. Prepare and submit the Development Plan and Technical Design described below. Implementation should begin only after this plan has been reviewed.

Your plan should be concise but technically specific. It must show your own reasoning and should not simply repeat the requirements in this document.

1. Proposed architecture and module/folder structure.
2. Domain model and ERD: main entities, relationships, ownership, and important constraints.
3. Booking overlap and concurrency strategy, including the exact database-level protection you intend to use.
4. Availability model: recurring schedules, exceptions, precedence, timezone approach, and slot-generation algorithm.
5. Booking state machine and allowed transitions.

6. Authentication and RBAC approach.
7. Redis strategy: what will be cached, key format, TTL, and invalidation events.
8. BullMQ strategy: queues/jobs, retry behavior, and how duplicate side effects will be prevented.
9. Idempotency strategy for booking creation.
10. API endpoint map (high level is enough at this stage).
11. Database indexing strategy for core queries.
12. Testing strategy including concurrency tests and important edge cases.
13. Docker/local development topology.
14. Implementation phases and the order in which you will build them.
15. Risks, assumptions, open questions, and anything you intentionally leave out of the first version.

**Plan quality matters.** A strong plan should make it possible for another developer to understand how you intend to protect data integrity and evolve the system before reading your implementation.

## 15. Suggested Development Phases

You may propose a better sequence in your Development Plan. The following is a recommended baseline, not a requirement to copy exactly.

| Phase   | Area                    | Expected Outcome                                                                       |
|---------|-------------------------|----------------------------------------------------------------------------------------|
| Phase 1 | Foundation              | Project setup, config, logging, PostgreSQL, Prisma, migrations, Docker, health checks. |
| Phase 2 | Identity & Access       | Users, authentication, JWT lifecycle, RBAC, security middleware.                       |
| Phase 3 | Booking Catalog         | Businesses, locations, services, resources, service-resource assignments.              |
| Phase 4 | Availability Engine     | Recurring schedules, exceptions, timezone rules, slot generation.                      |
| Phase 5 | Booking Core            | Booking creation, lifecycle, cancellation, overlap protection, transactions.           |
| Phase 6 | Operational Reliability | Idempotency, Redis cache/invalidation, BullMQ worker/jobs, audit log.                  |
| Phase 7 | Quality & Handover      | Swagger, seed scenarios, automated tests, concurrency suite, README, final report.     |

## 16. Edge Cases You Are Expected to Think About

- A service ends exactly when the next booking starts: is that allowed?
- A working period is shorter than the service duration.
- A resource is deactivated after future bookings already exist.
- A service duration changes after bookings were created.
- A customer retries the same request because the network timed out.
- A booking is cancelled while a cached availability response still exists.
- Two requests attempt overlapping but not identical time ranges.
- An availability exception conflicts with the normal weekly schedule.
- Daylight-saving/timezone changes affect local booking times.
- A background job is executed more than once.
- An admin tries to cancel or complete a booking in an invalid state.
- A customer attempts to access another customer's booking.

## 17. Evaluation Criteria

The project will be reviewed as an engineering system, not only as a list of completed endpoints.

| Evaluation Area                | What Will Be Reviewed                                                                          |
|--------------------------------|------------------------------------------------------------------------------------------------|
| Architecture & Domain Modeling | Clean boundaries, reusable concepts, sensible relational design, low hard-coding.              |
| Correctness & Data Integrity   | Overlap prevention, transactions, authoritative server-side rules, safe lifecycle transitions. |
| Code Quality                   | Type safety, naming, modularity, error handling, maintainability.                              |
| Database Engineering           | Constraints, indexing, migrations, transaction strategy, query quality.                        |
| Testing                        | Coverage of important behavior, integration quality, concurrency proof, regression thinking.   |
| Security                       | Auth, RBAC, secrets, validation, safe responses, ownership checks.                             |
| Redis / Queue Design           | Useful cache strategy, invalidation, retry-safe jobs, no misuse as source of truth.            |
| Developer Experience           | Docker setup, README, Swagger, seeds, reproducible commands.                                   |
| Engineering Communication      | Quality of the Development Plan, documented decisions, assumptions, and final report.          |

## 18. Definition of Done

- The complete application starts locally from documented commands.
- Database migrations and seed data work from a clean environment.
- At least three different business domains are demonstrated using the same booking engine.
- Authentication and authorization rules are enforced.
- Availability and exception rules produce correct slots.
- Booking overlap protection remains correct under concurrent requests.
- Idempotent retries do not duplicate bookings.
- Redis invalidation keeps availability results consistent enough for the designed TTL strategy.
- Background jobs can be observed and tested locally.
- Automated tests pass consistently.
- Swagger/OpenAPI and README are complete enough for another developer to use the system.
- Known limitations are documented instead of hidden.

## 19. Final Note to Ahmed

The purpose of this project is not to build the largest possible system. The purpose is to demonstrate that you can design a correct, understandable, testable backend and make deliberate engineering decisions. Keep the scope controlled, protect the important invariants, and prefer a simple design that you can explain and test over unnecessary complexity.

**Your first submission is the Development Plan + Technical Design.**

**Do not start implementation until it has been reviewed.**